

AIQG — Traffic Attribution Per Agent Flow

Status: **Partially shipped** — locked identity model; gateway header/baggage plumbing, AgentContext, emitter promotion, /api/v1/events, and the Traffic Explorer shipped 2026-06-11 (aiqg-v5.24). The **Inferred attribution** section below is design-only. Owner: AIQG
Related: [AIQG-EXTENSION.md](#), aether-shared/data-models/aiqg/

Problem

AIQG today classifies each request by workflow_type and scores it (CLEAR), but it cannot answer **“which agent, and which agent run, produced this traffic?”** A single agent execution fans out into many gateway calls (planner → workers, ReAct tool loops, RAG sub-queries) spanning multiple workflow types. We want to **group gateway requests into the agent, the agent run/flow, the conversation, and the step that produced them**, and roll cost / CLEAR / avoidable-cost up to the **agent** and the **flow**.

A *flow* is *orthogonal* to workflow_type: one orchestration flow contains agentic + rag + summarization *steps*. This feature adds a grouping axis *above* the per-request classification we already have.

Prior art (informs the model)

Surveyed standards, observability tools, and identity protocols (deep-research, 2026-06; 21 verified claims). Consensus:

- **OpenTelemetry GenAI semantic conventions** (experimental) define the canonical layered scheme:
 - gen_ai.agent.id (machine id) + gen_ai.agent.name (human) + gen_ai.agent.version / .description — names the AGENT.
 - gen_ai.conversation.id — folds **session + thread into ONE id**.
 - the **trace** (trace_id) is the RUN; **spans** are STEPs, named by gen_ai.operation.name (invoke_agent, execute_tool, invoke_workflow), linked via trace_id/span_id/parent_span_id.
- **Observability tools converge** on the same layering with different names: LangSmith run_id+parent_run_id+trace_id and interchangeable session_id/conversation_id/thread_id; Langfuse 32-hex trace + 16-hex observation + free-form sessionId; Helicone pure-HTTP-header Helicone-Session-Id + Helicone-Session-Path ('/' tree) + Helicone-Session-Name.
- **Propagation**: SDK-managed trace context in libraries; **a gateway must use headers** (Helicone is the proven header-only model), optionally honoring **W3C traceparent** for trace/span correlation.
- **Trust**: today's attribution is **self-asserted** (caller picks the ids/names) everywhere. **Authenticated agent identity is the frontier** — AIP (Agent Identity Protocol: aip:web: / aip:key:ed25519: ids, Ed25519-signed identity docs, Invocation-Bound Capability Tokens via X-AIP-Token), Google A2A, WSO2 AI Gateway “AgentID”.
- **Gap**: patent freedom-to-operate is **unresolved** — 4 USPTO patents on gateway attribution/metering were located (US 12,483,411 / 12,547,677 / 10,924,326 / 11,785,115) but their claims were not verified. FTO due diligence is owed before this feeds billing.

Locked identity model

Four levels, named with OTel gen_ai.* where a standard attribute exists:

```
tenant
└─ agent          gen_ai.agent.id (+ gen_ai.agent.name, .version)  WHO
    └─ conversation gen_ai.conversation.id (session + thread folded) THREAD
```

```

└─ flow      flow_id (= W3C trace_id when present)      ONE RUN
    └─ step  step_id (= W3C span_id) + parent_step_id    ONE CALL

```

- A **step** is exactly today's request_event / response_event.
- A **flow** groups all steps sharing flow_id — one agent execution.
- **Conversation** groups flows in one thread; reuses/extends the existing session_id_inferred.
- **Agent** is the stable definition (maps to tas-agent-builder Agent.ID/Name).

Identity sources (asserted-when-available, inferred-otherwise)

Every event records identity_source so the dashboard never conflates a named flow with a guessed one.

Field	Primary source	Fallback
gen_ai.agent.id / .name	TAS-Agent-Id / TAS-Agent-Name header	agent surrogate (tier 5) → el
flow_id / step_id / parent_step_id	W3C traceparent (trace_id/span_id)	TAS-Flow-Id header → else i
gen_ai.conversation.id	TAS-Conversation-Id header	baggage session.id → existi
user_id	baggage user.id	(none → principal/IP subdivi
agent_version	TAS-Agent-Version header	omit

Trust path: ship self-asserted (matches the entire market); design the schema so an **authenticated** agent_id (AIP IBCT via X-AIP-Token, or a binding to the TAS-Auth token's claims, or WSO2-style AgentID) can slot in later without a migration. A future agent_identity_verified boolean + agent_credential_ref cover this.

Resolution ladder — attribution without client code changes

When clients **cannot be modified**, the cooperative tiers (1-2) go dark. Resolve top-down and stamp the tier that won (identity_source + identity_confidence). Key corrections from the original framing:

- The data-plane credential is the **AIQG token** (tas_qg_live_* → token_id, tenant, account, source_app, label), **not** a Keycloak JWT (Keycloak only authenticates the dashboard control plane). Token granularity is per-app/credential, so it does **not** scale to a user spanning many apps.
- The cross-app **user** key is **W3C baggage** (user.id / session.id / account.id — Datadog default keys), set once at the identity boundary and **auto-propagated by the existing APM/OTel layer** with no app-logic change. This is the missing identity carrier: traceparent gives flow/step correlation but no *who*; baggage gives the *who*.

1 Authenticated	AIP IBCT / token-bound agent_id		billing-grade future (FT0-gated)
2 Asserted	TAS-Agent-* headers / traceparent	high	client opt-in
	└─ baggage	baggage: user.id, session.id, account.id	high (self-asserted) ← cross-app USER key
		└─ propagated by existing Datadog/OTel APM — no app-logic change	
3 Linked	tool_call_id echo / prefix chain / content	HIGH (evidence-based)	← flow/step topology, no code
		└─ unforgeable-ish: the correlating token was generated BY US (or the vendor, through us	
4 Principal	AIQG token (token_id + source_app + label)	high	free, no code
5 Transport	truncated source IP / XFF	medium	subdivides a shared token
6 Fingerprinted	template+toolset+config signature → cluster	medium	agent TYPE identity, n
7 Behavioral	timing / motif / step-sequence clustering	low	
8 Unattributed	—		

identity_source $\in$ {authenticated, asserted, baggage, linked, principal, transport, fingerprinted, behavioral, unattributed} (the linked and fingerprinted values are additive — see **Inferred attribution** below); identity_confidence $\in$ [0,1] decays down the ladder.

Ladder subtlety: linked sits below asserted for *agent naming* but is arguably **stronger for flow/step topology** — a client can lie in TAS-Flow-Id, but it cannot fake having received call_abc123 from us. Rule: asserted wins for *who* (agent identity), linked wins for *shape* (flow/step parentage); when both exist and **disagree**, that mismatch is itself an anomaly signal (see Cross-checks below).

Inferred attribution — zero-cooperation identification (design)

Tagging (tiers 1-2) requires a developer or ops team to mark each client. For unmodified clients those tiers go dark, and the original ladder dropped straight to low-confidence heuristics. This section closes that gap: **the gateway can infer flow topology deterministically and agent identity statistically from the traffic itself**, with no client changes.

Key insight — the protocol echoes our own output back at us. Chat completions are stateless: the client must re-send the entire conversation history on every turn, so every response we serve reappears verbatim inside a later request's messages array. This is the WAN-dedupe / byte-caching position (Bluecoat, Riverbed): a middlebox that indexes what it serves can recognize it coming back and assign identity to the stream — except here the “replay” is guaranteed by the API shape, not just statistically likely.

A. Deterministic flow-instance linkage (identity_source=linked)

These are not heuristics — the correlating token was generated by us (or the vendor, through us), so a match is proof:

1. **tool_call_id echo.** When a response we serve contains tool_calls, the vendor mints tool_call_ids. The agent's next request MUST include role=tool messages carrying those exact ids (Message.ToolCallID, already parsed). Index every served tool_call_id $\rightarrow$ (flow_id, step_id) in a short-TTL store (Redis, minutes-hours). When an id reappears: that request is provably the **next step of the same flow**, and the request whose response carried the id is its **parent step**. Yields flow_id, step_id, parent_step_id — the full ReAct loop reconstructed — with zero cooperation and zero ambiguity.
2. **Prefix chaining (assistant-message echo).** Multi-turn without tools: request N+1's messages = request N's messages + our response + one new user message. Keep a rolling per-message hash chain; map hash(prefix + served_response) $\rightarrow$ conversation_id. A new request whose prefix hash matches = **same conversation, next turn**. Prior art that prefix identity $\equiv$ conversation identity: Anthropic/OpenAI prompt caching and vLLM prefix caching key on exactly this.
3. **Content-propagation graph (the generalized byte cache).** Planner $\rightarrow$ worker orchestration: the planner's response contains task text that reappears as *user messages* of subsequent requests — different process, different model, different toolset, **no shared context for any header scheme to propagate**. Content-defined-chunk (Rabin fingerprint — robust to byte shifts) every served response into a TTL'd fingerprint index; chunk and look up every incoming request. Partial overlap = a **data-flow edge**: parent flow $\rightarrow$ child sub-agent. Suppress boilerplate with IDF weighting — chunks seen across many principals/templates (“You are a helpful assistant”, shared RAG corpus chunks) carry no attribution weight, the classic too-common-dictionary-entry problem from byte caching.

B. Agent-type fingerprinting (identity_source=fingerprinted)

Tier A answers *which run*; this answers *which kind of flow / which agent owns it*. Programmatic callers leak identity through everything they don't randomize. Signal catalog, roughly by leverage:

#	Signal
1	Toolset hash — hash(sorted tool names + param schemas)
2	System-prompt template — Drain-style log-template mining (static skeleton, variable slots ignored)
3	User-message template
4	response_format JSON schema
5	Few-shot exemplar blocks
6	Scaffolding idioms — delimiters, ### Instructions, <document> tags, Thought:/Action:/Observation
7	Serialized payload vocabulary — JSON keys inside stringified context ("customer_ticket_id":)
8	Config tuple — (model, temperature, top_p, max_tokens, stop, seed)
9	Step-archetype sequences
10	Call-graph topology — fan-out width, depth, parallelism, loop counts
11	Token-count profile — in/out size vector along the flow
12	Periodicity
13	Inter-step latency — gap between our tool_calls response and the next request = the tool's real execution time

No single proxy is the identity — the identity is a **cluster in the joint signal space** (toolset + templates weighted heavily; config and shape as tie-breakers), always scoped per (tenant, principal) first so two customers running the same open-source agent never collide. This upgrades AgentContext.AgentSurrogateID from the naive hash(source_app+prompt_hash+toolset) to the ensemble cluster id.

C. Version lineage

When a signature changes *partially* (template edited, toolset identical), do not mint a new agent — link it as a new **version of the same lineage** (MinHash Jaccard above threshold → same agent, signature_version++). Identity survives prompt iteration, the main failure mode of naive prompt-hashing.

D. Agent registry — humans name clusters, not flows

The answer to the tagging burden: inference does 100% of the *grouping*; humans only *label the groups*, once. New inferred table (dashboard-be):

agent_registry: surrogate_id, signature components (toolset_hash, template_id, config tuple), version lineage, first_seen / last_seen, sample prompts (hashes + redacted excerpts), human_label (nullable), linked TAS-Agent-Id (nullable, set when asserted traffic matches)

Dashboard shows "14 distinct agents detected (3 named, 11 unnamed)"; one click christens a cluster ("this is the Invoice Processor") and every future matching flow auto-attributes. Tagging cost drops from *per-flow, per-dev, forever* to *per-agent-type, once*. Cooperatively tagged traffic (Phase 3 agent-builder headers) doubles as **labeled seeds** that calibrate the clustering — semi-supervised: the tags teach the inferrer.

E. Cross-checks and anomalies

When an *asserted* TAS-Agent-Id arrives on traffic whose *inferred* signature belongs to a different cluster, surface the mismatch: misconfigured headers at best, **agent impersonation**

at worst. The inference tier audits the cooperative tier — feed disagreements to the Security report.

F. Practicalities

- **State:** TTL'd LRU fingerprint store (Redis) — `tool_call_id` index minutes-hours; prefix chains hours; template/registry store long-lived and tiny. Conversations age out like cache entries.
- **Privacy:** store fingerprints/hashes only, never content — this tier is *more* privacy-preserving than the baggage tier (nothing user-identifying extracted). Registry sample excerpts are redacted + opt-in.
- **Streaming:** hash the response as it streams through (we already proxy it); finalize the fingerprint at stream end.
- **Cost:** hashing/CDC is cheap and inline; embedding-based fuzzy cluster assist (if ever needed) runs async on the cold path.

G. Implementation order and validation

By leverage: **(1) tool_call_id echo + prefix chaining** (a Redis map keyed on hashes of data already flowing through the gateway — deterministic, small) → **(2) toolset hash** → **(3) system/user template mining (Drain)** → **(4) config tuple** → **(5) step-archetype sequencing + registry UI** → **(6) content-propagation chunking** (most novel, most speculative — also absent from all surveyed prior art, so worth a defensive-publication / patentability look of our own alongside the FTO pass).

Validation is free: `cmd/demo-traffic` already synthesizes multi-step agent profiles with known ground truth — run it **untagged** and measure how much flow topology and agent grouping the inference recovers (precision/recall per tier) before touching production paths.

Header contract (new)

Added to `internal/middleware/aiqq_headers.go` `canonicalHeaderNames` (parsed, validated, and **stripped before the vendor** via the existing `StripFromOutbound`):

Header	Meaning	Validation
TAS-Agent-Id	stable agent id	≤128 chars
TAS-Agent-Name	human-readable agent name	≤128 chars
TAS-Agent-Version	agent version	≤64 chars, optional
TAS-Flow-Id	explicit run/flow id (when not using traceparent)	≤128 chars
TAS-Conversation-Id	conversation/thread id	≤128 chars

Standard headers also honored: **W3C traceparent** (read, not stripped → `flow_id=trace_id`, `step_id=span_id`), `tracestate` (passthrough), and **W3C baggage** — parsed for a configurable key list (default `user.id`, `session.id`, `account.id`, the Datadog defaults) and **stripped before the vendor** (`StripFromOutbound`). `baggage` is the no-app-change cross-app user key: auto-propagated by the caller's existing Datadog/OTel APM, self-asserted trust. `user.id` MUST be opaque/pseudonymous (no PII). Existing `X-Session-ID` / `X-Request-ID` behavior unchanged.

Event schema additions (additive only)

Per the AIQG non-breaking constraint, add a new optional sub-struct to `pkg/aiqq/events/event.go` `RequestEvent` and `ResponseEvent`:

```

type AgentContext struct {
    AgentID      string `json:"agent_id,omitempty"`           // gen_ai.agent.id
    AgentName    string `json:"agent_name,omitempty"`       // gen_ai.agent.name
    AgentVersion string `json:"agent_version,omitempty"`    // gen_ai.agent.version
    AgentSurrogateID string `json:"agent_surrogate_id,omitempty"` // tier-5: hash(source_app+o
    ConversationID string `json:"conversation_id,omitempty"`  // gen_ai.conversation.id (o
    UserID       string `json:"user_id,omitempty"`         // baggage user.id – cross-a
    FlowID       string `json:"flow_id,omitempty"`        // run/trace
    StepID       string `json:"step_id,omitempty"`        // span
    ParentStepID string `json:"parent_step_id,omitempty"`
    FlowStepSeq  int    `json:"flow_step_seq,omitempty"`   // 1-based order within flow
    PrincipalID  string `json:"principal_id,omitempty"`    // AIQG token_id / source_ap
    ClientIPHash string `json:"client_ip_hash,omitempty"`  // tier-4, truncated + deplo
    IdentitySource string `json:"identity_source,omitempty"` // authenticated|asserted|ba
    IdentityConfidence float32 `json:"identity_confidence,omitempty"` // [0,1], decays down the
    ResolvedBy    []string `json:"resolved_by,omitempty"`    // e.g. ["baggage.user.id",
    IdentityVerified bool    `json:"agent_identity_verified,omitempty"` // future: credential-b
}

```

Emitter promotion (emitter.go): promote agent_id, agent_name, user_id, flow_id, conversation_id, principal_id, identity_source as top-level Loki fields so the dashboard can | json | user_id="..." / group by (agent_id) / by (user_id) / by (flow_id) — mirrors how workflow is already promoted.

Work breakdown (per repo)

1. **tas-llm-router (gateway)** — header parsing (TAS-* + traceparent + **baggage** for user.id/session.id/account.id, configurable keys, stripped before vendor) + flow/step derivation + resolution-ladder fallback (baggage → principal/token → truncated IP → behavioral) + AgentContext on events + emitter field promotion. The core. The baggage tier is the no-app-change path for un-instrumented clients (their existing Datadog/OTel APM already propagates it).
2. **tas-agent-builder** — start sending TAS-Auth (needs a service account/token so its calls enter AIQG Path A at all) **plus** the new headers mapped from Agent.ID→TAS-Agent-Id, Agent.Name→TAS-Agent-Name, AgentExecution.ID→TAS-Flow-Id, AgentExecution.SessionID→TAS-Conversation-Id, and a per-call TAS-Flow-Step seq. Without this, agent-builder traffic stays invisible.
3. **aiqq-dash-board-be** — new aggregation: GET /api/v1/metrics/agents (per-agent rollup), GET /api/v1/metrics/users (per-user_id rollup, the cross-app axis), and GET /api/v1/flows + GET /api/v1/flows/{flow_id} (flow list + step tree drill-down) + GET /api/v1/events (raw per-event list + live tail for the Traffic flow-inspector). Loki group-by user_id/agent_id/flow_id; TimescaleDB scope_type ∈ {user,agent,flow} rollups for the fast path.
4. **aiqq-ui** — “Traffic by Agent” view (table/cards per agent) and a **flow drill-down** showing the step tree (parent/child) with per-step workflow_type, cost, latency, CLEAR, and tags — the agentic equivalent of a trace waterfall.
5. **tas-spark-jobs / TimescaleDB** — add scope_type ∈ {agent, flow} rollups to aiqq.metrics_* so per-agent dashboards don't hit Loki.
6. **aether-shared/data-models/aiqq/** — new agent-context.md model doc + updates to request-event.md / response-event.md / inferred-labels.md (cross-repo PR lands together per repo convention).
7. **cmd/demo-traffic** — extend the generator to stamp agent_id, agent_name, flow_id, conversation_id, step so the new agent/flow views can be validated immediately against demo data (define a few demo agent personas, each running multi-step flows across

workflow types). Add an **--untagged mode** that strips all cooperative identity so the inferred-attribution tiers can be scored against known ground truth.

8. **tas-llm-router (inference engine)** — the Inferred attribution design: Redis linkage store (tool_call_id index + prefix chains), toolset/ template/config fingerprinting, agent registry + cluster naming in aiqg-dashboard-be/aiqg-ui. Phased per section G; ships behind its own config flag, additive to the event schema.

Phasing

- **Phase 1 — Gateway plumbing:** header contract + AgentContext + emitter promotion + inference fallback. Verifiable via demo-traffic (#7) and Loki group-by, with zero other-repo changes.
- **Phase 2 — Dashboard read path:** aiqg-dashboard-be endpoints + aiqg-ui agent view + flow drill-down. Validated against demo data.
- **Phase 3 — Real source:** instrument tas-agent-builder (service token + headers) so live agent traffic flows through with accurate identity.
- **Phase 4 — Scale + trust:** TimescaleDB/Spark per-agent rollups; then the authenticated-identity upgrade (AIP/IBCT or token-bound agent_id), gated on the **patent FTO** outcome.

Open items

- **[Queued] Thorough baggage research pass** — cross-vendor adoption (OTel / Datadog / Honeycomb / Grafana), spec size limits, and how user.id seeding works per stack; update AIQG-AGENT-IDENTITY-RESEARCH.md (which flagged W3C Trace Context/Baggage as under-surveyed). Confirmed so far: W3C baggage header, Datadog default keys user.id, session.id, account.id, auto-propagated via OTel-compatible headers (default propagation style datadog, tracecontext, baggage).
- **Source-IP capture** is deployment-mode gated: ip_capture_mode: full | minimized | off (default minimized for the product schema; internal / self-hosted may set full = truncated + raw with a short-TTL deletion job). Internal service/pod IPs (RFC1918) are not personal data; human/end-user IPs (via XFF) are, even internally — and the shared schema is inherited by customer deployments.
- **Patent FTO pass** (US 12,483,411 / 12,547,677 / 10,924,326 / 11,785,115 and neighbors) before any billing use of per-agent metering. Extend the pass to cover the **content-propagation graph** (section A.3) — none of the surveyed prior art does it, so evaluate both FTO *and* our own defensive-publication / patentability position.
- **Inferred-attribution validation** — run cmd/demo-traffic --untagged against the inference tiers and record precision/recall per tier before enabling on production traffic.
- OTel GenAI conventions are **experimental** — pin the attribute names we adopt and revisit on each OTel semconv release.
- Decide whether flow_id defaults to W3C trace_id (OTel-native) or a TAS id when both traceparent and TAS-Flow-Id are present (proposed: explicit TAS-Flow-Id wins, else traceparent, else inferred).